

Reclamation — Design and Requirements

Reclamation

Contents

1	Building & Vehicles — Design & Status	3
1.1	1. Scope & principles	3
1.2	2. Core model (current)	3
1.3	3. Placement, rotation & content (current)	4
1.4	4. Physics & anchoring (current)	5
1.5	5. Vehicles (current: basic prototype)	5
1.6	6. Redesign — Expressive Grid + Parts (planned)	5
1.7	7. Roadmap (planned, dependency-ordered)	7
1.8	8. Status at a glance	7
1.9	9. Source map	8
1.10	Appendix — Foundational decisions (carried forward, still binding)	8
2	World Generation & Chunk Streaming — Design	10
2.1	Decisions (locked)	10
2.2	Reconciliation with prior material	10
2.3	Milestones	11
2.4	Performance	11
2.5	M1 code map	12
2.6	WorldGenTesting setup	12
3	Combat System — Design Specification	14
3.1	Architecture	14
3.2	Auto-Turret	14
3.3	Ammo	15
3.4	Enemy Placeholder	16
3.5	Factory Loop Integration	16
3.6	Setup Checklist	17
4	Item UI System — Design Specification	18
4.1	Architecture Overview	18
4.2	Key Files	18
4.3	Dual-Panel Layout	19
4.4	Drag-Drop Pattern (Click-to-Hold)	19
4.5	Machine Panel Sections	20
4.6	Chest Panel	20
4.7	Setup Checklist	21

4.8	Item Slot Sizes	21
4.9	Cursor State	21
4.10	StorageChestMachine API Additions	22
5	Input System — Design Specification	23
5.1	The context model	23
5.2	API	24
5.3	Bindings are rebindable	24
5.4	Behaviour change introduced by this refactor	24
5.5	Migration path to Unity’s Input System	24
5.6	Planned: Build & Combat modes — the R-key scheme	25
5.7	Source map	26
6	Reclamation — Use Cases & Scenarios	27
6.1	Gameplay fantasies	27
6.2	Control-flow scenarios	27

Chapter 1

Building & Vehicles — Design & Status

Authoritative source of truth for constructs — the block-grid building system, its physics/anchoring, and vehicles. Consolidates the prior per-system building/vehicle notes and supersedes the legacy [BuildingSystem_UML.md](#).

Status is marked **(current)** — implemented and in the codebase — or **(planned)** — designed here, not yet built.

1.1 1. Scope & principles

- **Constructs unify bases and vehicles.** A construct is a group of face-connected blocks. A terrain-anchored construct is a base; a released one with a seat + propulsion is a vehicle. Same data model, same physics, same building tools.
 - **Strict sim/view split.** All logic is pure C# in `Assets/Simulation/` (no `UnityEngine`); `Assets/View/` renders it and feeds input. Deterministic integer grid.
 - **Cell = 0.5 m** today (`ViewConstants.CellSize`); vehicles will get a finer *per-construct* grid (planned, §6C).
-

1.2 2. Core model (current)

Type	Key fields
<code>BlockDefinition</code> (immutable)	<code>Id</code> , <code>Category</code> , <code>FunctionalType</code> , <code>SizeX/Y/Z</code> (cells), <code>Mass</code> , <code>power</code> , <code>cost</code> , <code>Ports</code> , <code>Params</code>
<code>Block</code> (instance)	<code>Definition</code> , <code>ConstructId</code> , <code>GridPosition</code> (integer), <code>RotationSteps</code> (0–3, 90° about Y), <code>IsOnTerrain</code> , <code>Durability</code> , runtime state

Type	Key fields
Construct	BlockIds, Cells (occupancy map), IsAnchored, ManualUnanchored, IsPilotable, Type

- **Occupancy map** — `Construct.Cells` : `Dictionary<GridPos,int>` maps every filled cell → its block id (a multi-cell block registers one entry per cell). Maintained by `Simulation` on place/remove/split/merge. Gives $O(1)$ overlap + adjacency, so connectivity flood-fill is $O(\text{cells})$ not $O(\text{blocks}^2)$.
- **Overlap rejection** — `Simulation.PlaceBlock` returns `null` if the footprint is occupied; `Simulation.CanPlace(...)` exposes the same test. *No ghost-validity feedback yet — the click just no-ops; wiring `CanPlace` into `GhostBlock` is a follow-up.*
- **Connectivity** — placement joins a block's construct; removal (`RemoveBlock`) flood-fills the remainder and splits disconnected components into new constructs (returns the new ids).
- **Unity structure** — one `ConstructView` parent per construct (holds the `Rigidbody` when floating); `BlockView` children hold colliders. Unity compounds child colliders automatically.

1.3 3. Placement, rotation & content (current)

- **Placement** — `BlockPlacer`: terrain click (free XZ snap, new construct) vs block-on-block (snap to the hit face plane, join that construct). `BlockDismantler`: right-click removes (100% material return).
- **Rotation** — 90° about Y only (`RotationSteps` 0–3); odd steps swap the X/Z footprint. *(Expanded to 24 orientations in the redesign, §6A.)*
- **Designer blocks** — author blocks in the Editor without code:

File	Role
<code>BlockDefinitionSO</code> (view)	Inspector block → <code>ToDefinition()</code> pure-C# <code>BlockDefinition</code> + a <code>Prefab</code>
<code>BlockLibrary</code> / <code>BlockLibraryLoader</code>	SO array; registered on <code>GameManager</code> <code>Awake()</code>
<code>BlockRegistry</code> (static)	Merges code <code>BlockCatalogue.All()</code> + SO blocks; <code>All/GetById/GetPrefab</code>
<code>BlockViewFactory</code> (static)	Spawns the designer prefab for a block id, else a sized primitive cube

`BuildMenu` lists `BlockRegistry.All()`; `SaveLoadManager` resolves defs via it. Zero-setup safe (no library → code blocks only). **Today each block is its own `GameObject`** (prefab or cube) — this is what the procedural mesher replaces (§6G).

1.4 4. Physics & anchoring (current)

Space Engineers-style: grounded = static, cut loose = dynamic.

- `Block.IsOnTerrain` is set when a block is placed on terrain (terrain click, or a physical `Physics.CheckBox` against `BlockPlacer._terrainMask` so a base built into a hillside also anchors). `Construct.IsAnchored` is recomputed by `Simulation.RecalcAnchor()`: `IsAnchored = !ManualUnanchored && (any member block IsOnTerrain)`.
 - `ConstructView.ApplyPhysics()`: **anchored** → no Rigidbody (static compound collider); **floating** → Rigidbody (mass = Σ `BlockDefinition.Mass`, gravity on, Continuous, drag 0.1, angularDrag 0.5). Called after removal/split and per construct on load.
 - Save stores `isOnTerrain` and `manualUnanchored` per construct; physics is rebuilt on load.
-

1.5 5. Vehicles (current: basic prototype)

A vehicle is a released (non-anchored) construct the player pilots.

- **Blocks** — `PilotSeat` (`Seat`) and `Thruster` (`Propulsion`), `BlockCategory.Vehicle` (a **Vehicle** tab in `BuildMenu`). Params: `ThrusterParams { ThrustKN, ExhaustDir }`, `SeatParams { Facing }`.
- **Classification** — `ConstructSystem.Reclassify()` (ticked from `Simulation.Update()`) sets `IsPilotable = hasSeat && hasPropulsion` and `Type`.
- **Release** — `Construct.ManualUnanchored` overrides anchoring; entering a seat releases the craft and `ApplyPhysics()` gives it a Rigidbody. Persisted in saves.
- **Pilot** — `VehiclePilot` (on `Player`): **F** enters/exits a seat, disables `PlayerController` + build/interact input, parents the player to the construct, swaps the camera, draws a hint.
- **Drive** — `VehicleController` (on `ConstructView`): WASD translate, Space/Ctrl up/down, Q/E roll, mouse yaw/pitch as torque; `_hoverAssist` cancels gravity for easy flight.

Deliberate simplifications (revisit): thrust is **not power-gated** (thrusters draw no power; the `OperatingRate` hook is a TODO); **total-thrust** model, not directional summing (`ExhaustDir/Facing` unused); mouse-torque steering (no gyroscope); save stores only Y rotation (a craft saved mid-flight loses pitch/roll). Setup: add `VehiclePilot` to the `Player`.

1.6 6. Redesign — Expressive Grid + Parts (planned)

Why. The 0.5 m axis-aligned cuboid grid with 90°-only rotation can't express detailed builds — the driving example is a ~1.5 m sportscar with curves, angled panels, headlights, mirrors, and **suspension**. Keep the grid for structure; add expressiveness + a parts layer. Preserve occupancy, connectivity, physics, functional blocks, the designer pipeline, save/load.

Two problems, layered: **body resolution & shape** (finer grid + shaped blocks + full orientation) and **functional/cosmetic parts** (an attachment layer; wheels/suspension are articulated — jointed parts, never grid cells).

1.6.1 A. Full 24-orientation rotation (*foundational*)

Replace `Block.RotationSteps` (0–3) with `Block.Orientation` (0–23, the cube rotations) so slopes/corners face any way. Generalise the occupancy footprint (AABB of the rotated size box; a $1 \times 1 \times 1$ detail block is always 1 cell). Touches sim footprint, view, placement controls (add pitch/roll), and save/load.

1.6.2 B. Shaped blocks (procedural)

Slope, wedge, inner/outer corner, rounded, half-slab, etc. — occupy their cell(s), render a sub-shape (SE armor model). **Generated procedurally, not prefabs** (§G): each shape is a geometry template + a 6-face solidity mask, both rotated by (A); blocks carry a material id. **Split:** structural/armor/shaped blocks are procedural; **functional** blocks (thrusters, machines, seats, turrets) and **parts** keep authored prefabs. `BlockDefinition` gains a render mode — `Procedural(shape, material)` or `Prefab`.

1.6.3 C. Small grid (per-construct cell size)

Add `Construct.CellSize` (0.5 m default; e.g. 0.125 m “small”). Block `GridPos/size` stay integer cells, so occupancy/connectivity are unchanged — only the world scale differs. View scales by `construct.CellSize`. A construct is large- or small-grid at creation; true SE mixed grids (small welded onto large) deferred.

1.6.4 D. Part layer + surface fixtures

- `PartDefinition` (sim): Id, category (Cosmetic / Light / Wheel / ...), mass, params (e.g. `WheelParams`), + a view prefab. `Part` (sim): Id, `DefId`, `HostBlockId`, local pose. Parts do **not** occupy cells; they live on the construct and are removed with their host.
- **Wheels/suspension** = a `WheelCollider` (or `Rigidbody` + `ConfigurableJoint`) on the construct’s `Rigidbody` at the part pose; drive/steer/brake from a `VehicleWheelController`. This is what makes it an actual car (thruster flight is already done, §5).
- **Functional surface fixtures** — parts placed **anywhere on a block’s surface** (off-grid, free rotation), e.g. a control panel. Each carries a **collider hitbox**; placement respects it via two paths: block block uses the integer occupancy map; anything involving a fixture uses a physics `OverlapBox`. Fixture overlap is validated in the **view** (physics), not the sim — fixtures are detail, not structure. Save/load parts as (`defId`, `hostRef`, `localPose`).

1.6.5 E. Symmetry tool

A mirror plane across the construct centreline; placing a block/part also places the mirrored counterpart (mirrored pose + orientation; left slope → right slope). Near-mandatory for cars.

1.6.6 F. Fine-build tooling (making a small grid usable)

Decouple grid resolution from block size — a 10 cm grid means you *can* place at 10 cm, not that you place 10 cm blocks (most of a body is a few big panels). Plus **drag tools** (line / face / box, filled/hollow; batched place/remove), **blueprints** / **copy-paste** + **stamps**, and a later **surface brush**.

1.6.7 G. Block meshing — procedural, face-culled (*perf gate*)

Structural blocks are not per-block prefab cubes. A **construct mesher** (the blocky analog of the voxel Surface Nets mesher) emits geometry from the occupancy/block data: - **Hidden-face culling (Minecraft-style)**: per block, emit only faces the neighbour cell doesn't fully occlude (O(1) via `Construct.Cells`); each shape's 6-face solidity mask (rotated by orientation) makes shaped-block occlusion correct. - **Chunked + incremental**: subdivide a construct into mesh chunks; an edit re-meshes only the affected chunk(s). Boundaries need neighbour solidity (as with voxels). - **Materials** → **submeshes/atlas**; prerequisite for **greedy meshing** (later): merge coplanar same-material faces into large quads (mostly full/slab faces; slopes don't merge). - **Two hard consequences**: (1) **render mesh physics collider** — a concave culled mesh can't go on a *moving* Rigidbody, so dynamic constructs use **compound box colliders** (per block / greedy-merged) while static bases may use a concave MeshCollider; (2) **block picking** — with no per-block GameObject, raycast the combined mesh and convert hit → cell → block (Minecraft-style), touching BlockPlacer/BlockDismantler/BlockInfoHUD.

1.7 7. Roadmap (planned, dependency-ordered)

1. **24-orientation rotation** (§6A) — sim footprint + view + placement + save.
2. **Construct mesher** (§6G) — face-culled full cubes; render mesh + compound box colliders + mesh-based picking. Replaces per-block prefab cubes for structural blocks.
3. **Shaped blocks** (§6B) — slope/corner/slab templates + occlusion masks, on (1)+(2).
4. **Small grid** (§6C) — per-construct cell size (the mesher absorbs the block count).
5. **Drag tools** (§6F) — line/face/box, filled/hollow.
6. **Part layer + surface fixtures** (§6D) — cosmetic/functional, collider hitbox, save.
7. **Wheels + suspension** (§6D) — `WheelCollider` + `VehicleWheelController`, into piloting.
8. **Greedy meshing** (§6G) — coplanar same-material merge.
9. **Blueprints / copy-paste** (§6F).
10. **Symmetry tool** (§6E).

Tracks: [1→2→3→4] body & rendering; [6→7] function (wheels); [5, 9, 10] tooling/UX; [8] perf follow-up. Vehicle **thruster flight is already shipped** (§5); its pending items (power-gated thrust, directional summing, gyroscope, HUD) fold in alongside.

1.8 8. Status at a glance

Area	Status
Construct/Block model, occupancy map, connectivity split/merge	current
Placement (terrain / block-on-block), overlap rejection, dismantle	current
90° Y rotation	current (→ 24-orientation planned)

Area	Status
Designer blocks (SO → registry → prefab/cube per block)	current (→ procedural mesher planned)
Terrain anchoring + grounded/floating physics	current
Vehicles: seat/thruster, classification, release, pilot, thruster flight	current (basic)
Power-gated thrust, gyroscope, vehicle HUD	planned
24-orientation, procedural face-culled meshing, shaped blocks	planned
Small grid, drag tools, parts + surface fixtures, wheels/suspension	planned
Greedy meshing, blueprints, symmetry	planned

1.9 9. Source map

Area	Files
Core model	Simulation/Block.cs, Construct.cs, BlockDefinition.cs, Support/GridPos.cs, Tables/*
Placement/connectivity/occupancy	Simulation/Simulation.cs, Systems/ConstructSystem.cs
Build/place view	View/BlockPlacer.cs, BlockDismantler.cs, Hotbar.cs, BuildMenu.cs, GhostBlock.cs, ConstructView.cs, BlockView.cs
Designer blocks	View/BlockDefinitionSO.cs, BlockLibrary.cs, BlockLibraryLoader.cs, BlockRegistry.cs, BlockViewFactory.cs
Physics/anchoring	View/ConstructView.cs, BlockPlacer.cs, BlockDismantler.cs, SaveLoadManager.cs
Vehicles	Simulation/Params/{Thruster,Seat}Params.cs, Systems/ConstructSystem.cs, View/VehiclePilot.cs, VehicleController.cs

1.10 Appendix — Foundational decisions (carried forward, still binding)

Decision	Detail
Cell size	0.5 m; one shared <code>CellSize</code> constant (per-construct override planned).
Construct = parent GameObject	<code>ConstructView</code> parent + optional Rigidbody; blocks are child GameObjects; colliders compound.
Positions	Integer <code>GridPos</code> in sim; <code>localPosition = GridPos * CellSize</code> in view.
Simulation isolation	Pure C#, zero <code>UnityEngine</code> , no <code>Vector3</code> in sim.
Connectivity	Face-adjacency (6 dirs) via the occupancy map; event-driven on place/remove, not ticked.
Construct split	New parent GameObject; <code>SetParent(worldPositionStays:true)</code> per block; cells transferred.
Rotation	90° steps about Y today (→ 24 orientations). Vehicles/freeform continuous is a separate future.
Terrain placement	Free-form, no global grid; snapping only against block faces.
Dismantle recovery	100% material return (V1).
Save references	Wires/belts key blocks by (<code>constructIndex</code> , <code>gridPos</code>) — survives id reassignment.

Chapter 2

World Generation & Chunk Streaming — Design

Voxel, deformable, on a spherical planet, streamed around the player. This supersedes the flat-2D world-streaming model in the prior SDD ([archive/space-crusade/clean_SDD.txt](#) §7) and folds in the prior `tempSpaceCrusade` cube-sphere planet prototype.

2.1 Decisions (locked)

- **Voxel, smooth (Space Engineers feel).** Terrain is a signed density field meshed as a smooth isosurface. Mesher = **Naive Surface Nets** (dual-contouring-lite), chosen over Marching Cubes for the SE-like smoothness and because it avoids the error-prone 256-row MC tables. Swappable behind `SurfaceNets`.
- **Shell, not full-depth.** Only a band around the surface is voxel data: diggable rock to `ShellDepth` below the local surface (then a solid floor), open air above. Generous enough for terrain variation and **oceans** (surface dips below `SeaLevel`), with depth reserved for underground enemy-nest **tunnels** — caves are **deferred** (added/polished later).
- **Spherical planet, player-centric frame (M3).** Authoritative data lives in **planet space**; Unity space is a per-frame rotation+translation that keeps the player near the origin, upright (+Y). This is the Unity-native equivalent of SE's doubles + Havok physics clusters (Unity Transforms are float-only, so we can't copy SE literally). Per-object radial gravity; physics only near the player.
- **Determinism.** World = pure function of (`seed`, `position`) + saved edits. Noise is a pure-C# `PerlinNoise` (the prototype used `Mathf.PerlinNoise`, which is 2D-only and platform-dependent). Save = seed + voxel edit deltas + constructs.

2.2 Reconciliation with prior material

Prior	Verdict
SDD §7 flat 2D chunks (<code>Vector2Int</code> , 16 m, SIM/VIEW radii)	Superseded for addressing (sphere needs (<code>face,u,v,h</code>)); kept : 32-cell/16 m chunk, dual-radius streaming, “only loaded chunks simulate”, save/evict.
Prototype noise (<code>PlanetShapeSettings</code> layers, <code>NoiseUtil.FBM</code>)	Reused — it’s the density source. Ported to pure-C# <code>PerlinNoise.Fbm</code> .
Prototype <code>CubeSphere</code> math	Reused at M3 for chunk addressing.
Prototype cube-sphere surface mesh (<code>PlanetVisualizer</code>)	Repurposed as the cheap far-LOD shell behind near voxels.
<code>PlanetSettings</code> (seed, radius 2 km, amplitude, sea level)	Kept as the planet config (→ a view-layer SO at M3, like our other SO→C# bridges).

The key insight that makes these fit: the voxel **density is the heightmap read as a field** — `density(p) = (surfaceHeight - distanceAlong)`, so the planet shape is identical to the prototype’s, just diggable. At M3 the flat height swaps for the radial one with no change to the mesher.

2.3 Milestones

- **M1 (done)** — single voxel region, Surface Nets mesh + collider, dig/add, from seeded noise; shell with ocean basins (caves deferred). Flat framing (sphere swapped in at M3).
- **M2 (done)** — streaming around a moving origin: **XZ column chunks** ($32 \times N \times 32$, full shell height — flat framing), view radius + unload hysteresis, `GameObject` pooling, per-frame build budget, dig/add rebuilds only affected chunks. Seam-free via **world-space vertices** (neighbours emit bit-identical coincident boundary geometry — no gaps, no z-fighting; single-ownership stitching is a later optimisation). The SDD’s larger **simulation** radius and 3D cube-sphere chunks arrive with M3/M4.
- **M3 — in progress (stage 1 done)**. Spherical voxel planet: radial density (same FBM, the “surface” is now a radius), 3D cube-chunk streaming around the target with deep-solid and open-sky chunks skipped (`VoxelWorld.ChunkOverlapsSurface`), and `RadialGravityBody` (per-object gravity toward the centre — the SE model). Rendered in planet space at the world origin (fine precision for a ~300 m–2 km test planet). **Stage 2**: the player-centric “rotate the planet underneath” frame + a sphere-walking controller (this is where the precision/upright payoff lands, and the riskiest part). **Stage 3**: far-LOD using the prototype’s cube-sphere surface mesh.
- **M4** — persistence (edit deltas), integrate constructs/ore/anchoring with voxel terrain.

2.4 Performance

- **Heightmap caching (done)**. `VoxelWorld.SampleDensity` evaluates the surface FBM once per XZ column, not per voxel (~90× fewer noise evals per chunk). This is the bulk of the cost and removes most of the streaming hitch.
- **Threading (next lever)**. Density + Surface Nets + normals are pure C#/struct math and run off the main thread; only the `Mesh/MeshCollider` apply is main-thread (collider

pre-bakeable via `Physics.BakeMesh`). The one hazard is reading `VoxelWorld._edits` while digging mutates it — snapshot per job. Cheap interim alternative: time-budget the build loop instead of a fixed `maxBuildsPerFrame`.

- **Burst + Jobs (planet scale, M3+).** The project already has burst/collections/mathematics. Porting the noise + mesher to Burst `IJobs` over `NativeArray` (+ `Mesh.MeshData`) is the endgame for a full planet — needs the voxel code in blittable form (no `Dictionary/List/delegates`; `NativeHashMap` for edits).

2.5 M1 code map

File	Layer	Role
<code>Assets/Simulation/World/PerlinNoise.cs</code>	<code>Sim</code>	Deterministic 3D Perlin + FBM (pure C#)
<code>Assets/Simulation/World/VoxelMaterial.cs</code>	<code>Sim</code>	Per-voxel material enum
<code>Assets/Simulation/World/VoxelWorld.cs</code>	<code>Sim</code>	Density field: flat + spherical, shell, oceans, materials, dig/add edits
<code>Assets/View/World/SurfaceNetView.cs</code>	<code>View</code>	Smooth isosurface mesher (robust winding)
<code>Assets/View/World/VoxelChunkView.cs</code>	<code>View</code>	Meshes a region → Mesh + MeshCollider (world-space verts)
<code>Assets/View/World/VoxelWorldDriver.cs</code>	<code>View</code>	M1 single-region harness: generate + mouse dig/add + sea plane
<code>Assets/View/World/VoxelChunkStreamer.cs</code>	<code>View</code>	M2: streams column chunks around a target — pool, budget, dig/add
<code>Assets/View/World/VoxelPlanetStreamer.cs</code>	<code>View</code>	M3: streams a spherical planet (3D chunks, surface-only)
<code>Assets/View/World/RadialGravityView.cs</code>	<code>View</code>	PlanetMath up/down + RadialGravityBody (gravity toward the centre)

Density convention: > 0 solid, ≤ 0 air; surface at 0. 1 voxel = 1 cell = 0.5 m (shares the block lattice). M1 simplifications to revisit: full-region rebuild per edit (M2 does partial per-chunk), placeholder lit material (vertex-colour materials later), `float` density (SE-style `sbyte` packing later).

2.6 WorldGenTesting setup

1. On an empty `GameObject` at the world origin **(0,0,0)**, attach one of: **VoxelPlanetStreamer** (M3 spherical planet), **VoxelChunkStreamer** (M2 flat streaming), or **VoxelWorldDriver** (M1 single region). Use one, not several. They emit world-space geometry, so keep the

GameObject at the origin. For the planet, the camera is auto-lifted to $\sim \text{Radius} + 10$ on Start (it would otherwise be buried in rock).

2. Ensure the scene has a **Main Camera** (the stream target + dig aim) and a **Directional Light**.
3. Press Play: terrain streams in around the camera. **Move the camera/player to load and unload chunks; left-click digs, right-click adds.** Tune seed/amplitude/sea level and viewRadius/maxBuildsPerFrame in the Inspector; component menu **Rebuild World** after a seed change. (If terrain looks inside-out, set the material's Render Face to Both.)

Chapter 3

Combat System — Design Specification

Factory-integrated auto-turret defence and enemy placeholder.

3.1 Architecture

Simulation layer (plain C#)	View layer (Unity)
Enemy	EnemyView (capsule, movement, death)
EnemySystem	EnemySpawner (wave management)
TurretMachine	TurretView (barrel, targeting, flash)
TurretParams	
ItemCatalogue (turret_round)	
RecipeCatalogue (craft_turret_rounds)	
BlockCatalogue (gun_turret)	

3.1.1 Sim/View split for spatial queries

Enemy world positions are stored on `Enemy.WorldX / WorldZ` (simulation layer) but **written by EnemyView** each frame. This keeps spatial data accessible to `TurretView` without a `FindObjectsOfType` call, while letting movement run at frame rate for smooth animation.

3.2 Auto-Turret

3.2.1 Block definition (`BlockCatalogue.GunTurret`)

Field	Value
Id	gun_turret

Field	Value
Size	$1 \times 2 \times 1$ cells
Power draw	20 kW
Input port	face NegZ (ammo belt)
Construction cost	4 iron_plate, 2 iron_gear, 1 circuit_board

3.2.2 TurretParams

Field	Default
Range	15 world units
ShotsPerSecond	1
DamagePerShot	25

3.2.3 TurretMachine (simulation)

- Input buffer: 1 slot, pre-configured for `turret_round`
- No recipe, no output buffer
- `CanFire` $\rightarrow$ powered + ammo present + cooldown elapsed
- `TryFire()` $\rightarrow$ removes 1 round, resets cooldown (1 / ShotsPerSecond sec)
- Tick: decrements cooldown; sets `Operating` (ammo present) or `Waiting` (empty)

3.2.4 TurretView (Unity)

- Built on top of the block cube via `BlockPlacer` after placement
- Barrel pivot child rotates in world Y toward the nearest in-range enemy (120°/s)
- Calls `TryFire()` each frame — fires the moment the machine is ready
- Calls `EnemySystem.Damage(id, damage)` on a successful shot
- Muzzle flash: point light enabled for 0.06 s

3.3 Ammo

3.3.1 Items

Id	DisplayName	Stack	Recipe
<code>turret_round</code>	Turret Round	200	<code>craft_turret_rounds</code>
<code>rifle_round</code>	Rifle Round	200	<code>craft_rifle_rounds</code>

3.3.2 Recipes (both Assembler)

Id	Inputs	Outputs	Cycle
<code>craft_turret_rounds</code>	2 iron_plate	20 turret_round	1.0 s
<code>craft_rifle_rounds</code>	1 iron_plate	10 rifle_round	1.0 s

3.4 Enemy Placeholder

3.4.1 EnemyView

- Spawned as a red capsule (`PrimitiveType.Capsule`)
- Moves toward `Camera.main` at `Enemy.Speed` units/sec (stops 1.2 m away)
- Colour fades from bright red → dark red as health decreases
- On death: white flash → scale to zero over 0.35 s → removes self from `EnemySystem`

3.4.2 EnemySpawner

Attach to any persistent `GameObject` (e.g. `GameManager`).

Inspector field	Default	Meaning
<code>SpawnInterval</code>	8 s	Seconds between waves
<code>SpawnsPerWave</code>	1	Enemies per wave
<code>MaxEnemies</code>	12	Live enemy cap
<code>MinSpawnRadius</code>	12 m	Inner ring boundary
<code>MaxSpawnRadius</code>	28 m	Outer ring boundary
<code>SpawnY</code>	1.0	Capsule centre Y (feet at Y=0)
<code>EnemyHealth</code>	100	HP per enemy
<code>EnemySpeed</code>	3.5 m/s	Movement speed

3.5 Factory Loop Integration

```
[Assembler: Turret Rounds]
  ↓ belt
[Gun Turret]    fires at enemy    EnemySystem.Damage()
  |
  (needs power from grid)
```

The turret draws 20 kW. Its `OperatingRate` is set by `PowerSystem` just like any other consumer — a power deficit throttles fire rate. If ammo runs out the block shows yellow (`Waiting`); if unpowered it shows red (`NoPower`).

3.6 Setup Checklist

1. **EnemySpawner** — add component to GameManager (or a CombatManager object). Tune SpawnInterval and MaxEnemies to taste.
2. **Gun Turret block** — add BlockCatalogue.GunTurret to your Hotbar definition so it can be selected and placed.
3. **Ammo supply** — place an Assembler, select “Turret Rounds”, run a belt from its output into the turret’s back face (NegZ port).
4. **Power** — connect the turret to the power grid; it draws 20 kW.
5. **ItemDefinitionSO** — create a `turret_round` and `rifle_round` definition in the ItemLibrary so ammo shows correctly in the inventory and chest UI.

Chapter 4

Item UI System — Design Specification

Satisfactory-style dual-panel item management for the Reclamation factory game.

4.1 Architecture Overview

Simulation layer (plain C#)	View layer (Unity)
ItemCatalogue	ItemLibrary (ScriptableObject)
ResourceItem	ItemDefinitionSO (per item)
ComponentItem	ItemLibraryLoader (MonoBehaviour)
WeaponItem	
ItemStack	ItemSlotWidget (plain C# helper)
ItemBuffer	DragDropController (MonoBehaviour singleton)
StorageChestMachine	
	PlayerInventory
	MachineInteractor dual-panel
	ChestInteractor

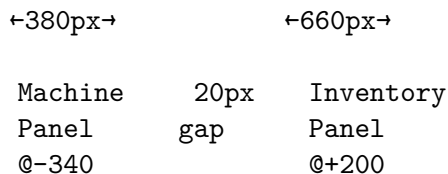
4.2 Key Files

File	Layer	Purpose
Assets/Simulation/Items/ItemCatalogue.cs	Simulation	Static item definitions (logic, stats)
Assets/View/ItemDefinitionSO.cs	View	Per-item icon + UI tint (ScriptableObject)
Assets/View/ItemLibraryView.cs	View	Catalogue of all ItemDefinitionSO assets

File	Layer	Purpose
Assets/View/ItemLibraryViewer.cs	Viewer	Registers ItemLibrary at startup (on GameManager)
Assets/View/ItemSlotWidget.cs	Widget	Reusable slot widget (background, icon, qty badge, button)
Assets/View/DragDropController.cs	Controller	Click-to-hold drag-drop singleton (on GameManager)
Assets/View/PlayerInventory.cs	View	Player inventory + gear slots
Assets/View/MachineInteraction.cs	Interaction	Machine config panel (buffer slots + recipe selection)
Assets/View/ChestInteraction.cs	Interaction	Chest contents panel (5×6 slot grid)

4.3 Dual-Panel Layout

When a machine or chest is opened, both panels appear side by side:



- Machine/chest panel: `anchoredPosition.x = -340` (DualLeftX)
- Inventory panel: `anchoredPosition.x = +200` (DualRightX)
- Standalone inventory (I key): `anchoredPosition.x = 0`

4.4 Drag-Drop Pattern (Click-to-Hold)

DragDropController manages state:

Nothing held:

Click slot → `take()` → hold the stack + show ghost

Something held:

Click slot → `place(_held)` → returns leftover

leftover.IsEmpty → fully placed → clear state

leftover == `_held` → fully rejected → cancel if same slot

else → partially placed / swap → update ghost

Right-click → cancel → source return callback → item back to origin

ESC → MenuManager closes panels → `ClosePanel()` → `CancelDrag()`

4.4.1 Callback Signatures

```
// Called by each slot on click
DragDropController.Instance?.HandleClick(
    widget,                                // null OK for gear/non-widget slots
    () => TakeFromSlot(),                  // Func<ItemStack>
    s => PlaceIntoSlot(s));                // Func<ItemStack, ItemStack>
```

4.5 Machine Panel Sections

Dynamic layout built per machine open (and rebuilt on recipe change):

```
[Title: "Iron Smelter"]
[
]
INPUT                                (if InputBuffer.Slots.Count > 0)
[ore slot] [...]
[Smelt Iron ]                        recipe / resource selection buttons
[Smelt Copper]
[
]
OUTPUT                                (if OutputBuffer.Slots.Count > 0)
[plate slot] [...]
[
]
[- Clear Recipe - ]
```

- Selecting a recipe rebuilds the panel immediately (no close)
 - Clicking clear closes the panel
-

4.6 Chest Panel

```
[Storage Chest]
[
]
[    ] 450 / 2000    capacity bar
[
]
[][][][][]          5-column × 6-row slot grid
[][][][][][]        30 slots max; sorted by qty desc
...
[+ 5 more types...] overflow indicator
```

- Items are sorted by quantity descending each frame
 - Slots are pre-allocated (30 widgets, shown/hidden)
 - Click slot → takes up to MaxStackSize into cursor
 - Place into chest → adds to storage (rejects if full)
-

4.7 Setup Checklist

1. **GameManager GameObject** — add components:
 - `UIRoot` (existing)
 - `ItemLibraryLoader` → assign your `ItemLibrary` asset
 - `DragDropController`
 2. **Player GameObject** — already has:
 - `MachineInteractor`
 - `ChestInteractor`
 - `PlayerInventory`
 - `Raycaster`, `Hotbar`
 3. **Item Library** — right-click in Project:
 - Create → Reclamation → Item Library → name it `ItemLibrary`
 - For each item: Create → Reclamation → Item Definition → set `Id` to match `ItemCatalogue` (e.g. `iron_ore`)
 - Assign icon sprites and tint colours per item
 - Add all `ItemDefinitionSO` assets to the library's `Items` array
-

4.8 Item Slot Sizes

Widget	Size
<code>ItemSlotWidget.Size</code>	52 × 52 px
<code>ItemSlotWidget.SlotGap</code>	4 px
Gear row height	40 px
Chest grid	5 cols × 6 rows = 300 px wide

4.9 Cursor State

Who manages cursor	Condition
<code>MachineInteractor</code>	machine panel open (dual mode)
<code>ChestInteractor</code>	chest panel open (dual mode)
<code>PlayerInventory</code>	standalone inventory (I key)
<code>MenuManager</code>	pause/settings menu

Inventory never touches cursor in machine/chest mode — the left-panel interactor owns it.

4.10 StorageChestMachine API Additions

```
// Take up to qty of itemId. Returns what was actually taken.  
public ItemStack TakeFromStorage(string itemId, int qty)  
  
// Add up to qty of itemId. Returns amount actually added.  
public int GiveToStorage(string itemId, int qty)
```

Chapter 5

Input System — Design Specification

All player controls live in one place: the static `GameInput` class (`Assets/View/GameInput.cs`). No other script calls `UnityEngine.Input` directly.

This was adapted from a centralized `PlayerInput` pattern in another project, with three deliberate improvements for Reclamation:

1. **No hard-wiring to a single controller.** `GameInput` exposes semantic signals that *any* system reads, rather than calling one controller's methods. “Interact” means different things on a machine, a chest, or a seat — each system decides.
2. **A context model, not two action maps.** Reclamation has more than Player/UI: gameplay, piloting, a panel, the build menu, paused. `GameInput.Context` is the single definition of “what is the player doing,” replacing the duplicated `MenuManager.IsOpen` / `BuildMenu.IsMenuOpen` / `VehiclePilot.IsPiloting` / cursor checks that used to be copy-pasted across components.
3. **Camera/look stays in the controllers**, not in the input layer.

5.1 The context model

`GameInput.Context` (computed each access from existing global state):

<code>MenuManager.IsOpen</code>	-> Paused	(pause / settings; cursor free)
<code>BuildMenu.IsMenuOpen</code>	-> BuildMenu	(cursor free)
<code>VehiclePilot.IsPiloting</code>	-> Piloting	(cursor locked; VehicleController drives)
<code>Cursor.lockState != Locked</code>	-> Panel	(machine/chest/inventory; cursor free)
otherwise	-> Gameplay	(on foot; cursor locked)

Components gate with one check instead of several:

// before - duplicated in many files:

```
if (MenuManager.IsOpen) return;  
if (BuildMenu.IsMenuOpen) return;  
if (Input.GetKeyDown(KeyCode.E)) TryOpen();
```

// after:

```
if (GameInput.Context != InputContext.Gameplay) return;    // or the relevant context  
if (GameInput.InteractDown) TryOpen();
```


The “Q opens the build menu while seated” class of bug disappears: adding a new context (e.g. Piloting) is a change in one place, not a hunt across components.

5.2 API

Signal	Binding	Notes
Move	WASD (Horizontal/Vertical, raw)	Vector2
Look	Mouse X/Y	Vector2 ; consumers apply their own sensitivity
Scroll	Mouse wheel	hotbar rotation
MousePosition	cursor screen pos	drag-drop ghost
PrimaryDown / SecondaryDown	LMB / RMB (down)	place/dismantle/wire/belt/drag-cancel
InteractDown	E	machine/chest open-close
InventoryDown	I	inventory toggle
BuildMenuDown	Q	build menu toggle
PilotDown	F	enter/exit vehicle
PauseDown	Esc	menu back/pause
JumpDown	Space (down)	jump on foot
AscendHeld / DescendHeld	Space / LeftCtrl	vehicle up/down
RollLeftHeld / RollRightHeld	Q / E	vehicle roll
SaveDown / LoadDown	F5 / F9	save / load
HotbarSlotDown	1–9, 0	returns 0–9 or <code>–1</code>
Context	—	InputContext enum

5.3 Bindings are rebindable

Keys are static **fields** (e.g. `GameInput.Interact = KeyCode.E`), not constants, so a future settings/rebinding screen can reassign them at runtime with no other code change.

5.4 Behaviour change introduced by this refactor

- **On-foot movement and look are now gated to the Gameplay context.** Previously the character could still walk (though not look) while a machine/chest/inventory panel was open. Now both stop in any non-gameplay context, which is the intended behaviour for a build/management game.

5.5 Migration path to Unity’s Input System

If rebindable-by-default UI and gamepad support are wanted later, only **GameInput** changes: it becomes the wrapper around a generated **GameControls** (from an `.inputactions` asset), keeping

the same public properties so the consuming components are untouched.

5.6 Planned: Build & Combat modes — the R-key scheme

*Design intent (not yet implemented; combat/weapons don't exist yet). Folded in from the original controls design doc. Bindings here will be reconciled with the live **GameInput** table above when weapons land — some differ (e.g. **Q** currently toggles the build menu; this scheme uses **Q** to deselect and **R** for weapon handling).*

Reclamation has two primary on-foot modes with fast, deliberate transitions — the guiding principle is **no mode should feel like a trap**: you can always reach for a weapon, and always get back to building.

Build Mode (default). Cursor locked; crosshair drives placement/interaction.

Input	Action
1–7	Select block type (ghost shown)
8 / 9	Wire Tool / Belt Tool (click endpoints; right-click cancels the pending link)
Q	Deselect / cancel current slot
Scroll	Rotate block (Y-axis, four 90° stops)
LMB / RMB	Place / dismantle
E	Interact with machine (opens config panel)
R (tap)	Draw last weapon → enter Combat Mode

Combat Mode. Entered by drawing a weapon; cursor stays locked; raycast targets.

Input	Action
1 / 2	Primary / secondary weapon
3–5	Heavy weapon (rocket, LMG, ...) — excluded from the R quick-draw
LMB / RMB	Fire / ADS or alt-fire
R (tap)	Reload
R (hold)	Holster → return to Build Mode
Shift / WASD / Mouse	Sprint / move / aim

The R key is the bridge between modes — one button, three context-sensitive actions:

Context	Input	Result
Build	tap R	draw last weapon → Combat
Combat	tap R	reload
Combat	hold R	holster → Build

Rationale: short press = offensive (draw/reload), long press = defensive (holster), so “R = weapon management” holds in both modes. Heavy weapons require deliberate selection (1–5) and **don’t** update the last-weapon memory, keeping the quick-draw fast and light. Walkthroughs of this in play are in [use-cases](#) (scenarios 1–4).

Open questions: Alt+Scroll for multi-axis block rotation; auto-restore the Belt/Wire tool slot after holstering vs. return to last block slot; tap/hold thresholds (~0.3 s / 0.5 s); sprint-while-firing (disable vs. accuracy penalty); configuring machines (E) without holstering.

5.7 Source map

File	Role
Assets/View/GameInput.cs	All bindings, semantic signals, <code>InputContext</code>
PlayerController, VehicleController	read <code>Move/Look</code> , apply sensitivity
Hotbar, BlockPlacer, BlockDismantler, WireConnector, BeltConnector	gameplay-context actions
MachineInteractor, ChestInteractor, PlayerInventory, DragDropController	panel actions
BuildMenu, MenuManager, SaveLoadManager, VehiclePilot	menu / global actions

Chapter 6

Reclamation — Use Cases & Scenarios

Concrete gameplay scenarios that guide design — *what it should feel like*, not specs. Touchstones for tuning. Collected from the original fantasies and controls design docs.

6.1 Gameplay fantasies

6.1.1 Dogfight airbrake (Top Gun)

The *Top Gun: Maverick* airbrake beat (~41:15) — Maverick decelerates hard so his pursuer overshoots and ends up in front of him — but here **you’re** the one being chased, by a flying alien you can’t shake. You pull the airbrake, slip behind it, and fire. A target feel for vehicle/flight combat: momentum-based maneuvers that reward timing over raw speed.

6.2 Control-flow scenarios

Walkthroughs of the planned Build/Combat mode controls and the R-key synergy (see [input](#)). [key] = input pressed.

6.2.1 1 — Pure building session (no threats)

[1] pick Small Cube → [Scroll] rotate → [LMB] place → [6] Electric Furnace → [LMB] place by the generator → [E] open config, pick Iron Ingot → [9] Belt Tool → [LMB] miner output → furnace input → [8] Wire Tool → [LMB] pole → generator. *Scroll rotates without changing slot; Belt/Wire share the right-click cancel idiom; E works whenever Build Mode’s cursor is locked.*

6.2.2 2 — Pure combat engagement

[1] primary → [Mouse] aim → [LMB] fire → [R tap] reload → [2] secondary → [LMB] clear → [R hold] holster → Build Mode. *Tap R reloads (an accidental tap during a hold won’t holster early); 1/2 swap without holstering; hold-R is the only way back to Build — deliberate, prevents mid-fight mode flips.*

6.2.3 3 — The ambush (*core synergy*)

Deep in Build Mode, mid-belt, an enemy spawns. [R] instantly draws your last weapon (the Belt Tool auto-cancels) → [Mouse] engage → [R tap] reload → enemy down → [R hold] holster → [9] re-select Belt Tool → [LMB] resume. *One keypress goes building → fighting; last-weapon memory means no weapon menu; heavy weapons are excluded so R always feels fast; hold-R returns just as fluidly.*

6.2.4 4 — Base defence wave

[E] mid-recipe on an Assembler → [E/Esc] close (cursor re-locks) → [R] draw → [LMB] engage from behind wall blocks → [3] rocket for an armoured enemy → [LMB] kill → [2] back to secondary → [R tap] reload → wave clear → [R hold] holster → [E] confirm recipe. *Switching to a heavy (3) mid-combat is fine — the R quick-draw exclusion only applies from Build Mode, and heavies don't update the last-weapon memory, so R still holds the secondary.*